GAME 2401 A3 Part 4 Reflection

Part 4: Development Workflow and Use of AI

GAME 2401 A3, House of Healing. Michael Hardwicke.

What shipped and what did not

Shipped: the run loop (Ready, Playing, Won, Lost) on a state machine driver, a day timer as the lose condition, quota delivery as the win condition, scene reload as restart, the completion of the input system migration, an entry document for the project, a comment purge, and one optimization.

Not shipped: the recipe authoring and validation tool from Part 3, and the finite ingredient stock that was supposed to give the day timer something to squeeze. Both were in the plan. Neither got built.

I am putting that at the top rather than at the bottom because the shape of the term is the thing worth reflecting on. The A2 feedback said the tooling shipped and the game did not. For A3 I inverted it: the game loop closed and the tooling did not. Going two for two on shipping one half of the assignment is a scheduling result, not a technical one.

The brief and the specification disagreed

The A3 brief opens with “The playable game is in place.” It was not. The A2 feedback had already recorded that there was no lose state, no timer, no fail pressure and no restart, and that `IState.cs` had been carried unchanged since A1 with zero implementations.

The resolution was not to pick one document over the other. Part 1 asks for core systems refactored so they are reusable, and a state machine that knows nothing about the game it is driving is that by definition. The closing instruction in the A2 feedback was to build the loop by hand and close it so a run can start, be won, be lost and be restarted. Those turned out to be one piece of work seen from two angles, so it got built once and it answers both.

`StateMachine.cs` and `IState.cs` contain no reference to chemistry, quota, ingredients or UI. Copying those two files into a different project and writing new `IState` implementations is the whole reuse story. The four game-specific states live in `Gameplay/States/` and are meant to be thrown away.

Where the AI boundary sat

The A2 feedback asked for the boundary to be reasoned from properties of the problem rather than stated as a policy. The property that decided it this term was **how many places the work offers to be checked.**

During the repository split, the assistant offered a single script that would clone, filter, repack and push in one pass. I declined it and ran the commands one at a time. A script like that produces one line of output at the end and gives you nothing to disagree with in the middle. Six separate commands produced five points where something could be inspected, and four of those inspections caught something.

That is the same reason the assistant was allowed near the state machine and was not allowed near the recipe rules. The state machine has a visible failure mode: you press the start key, either the day starts or it does not. A miswritten combination rule fails silently and looks identical to a correct one until someone plays for ten minutes.

Where the boundary moved this week, and I want this recorded rather than smoothed over: the plan said the `GameManager` would be written by hand and committed before any assistance touched it, specifically so the commit order would be the evidence for this section. That is not what happened. It was written with assistance on Friday night, and the reason was the clock and nothing else. The argument for hand-writing it was never that assistance would produce worse code, it was that the receipt would be free. Once the clock made it not free, it got traded. Naming the trade is worth more than a receipt I would have had to fake.

Five places the assistant was wrong

All five happened inside a single planning and repository session. None of them were caught by reading the output and finding it plausible.

1. **Planned around a constraint that did not exist.** An entire risk section was built on the belief that the lighting bake required a booked school VM and took a long time. Both were false, from a stale note in the assistant's own memory. The bake is about five minutes and the VM is available on demand. Correcting it moved a task from half a day with an external dependency to about an hour.
2. **Recommended the wrong repository.** It proposed moving the animation assignment into the tutorial-follow repo, having read that repo's Cinemachine version but never asked what the repo was for. It is reference material for learning, not a submission target.
3. **Predicted `.git` would land at 680 MB. It landed at 468 MB.** The cause was adding a working-tree figure, 317 MB of Maya files sitting on disk, to a packed-repository figure of 366 MB. Those are different units. Maya packed down to about 102 MB.
4. **Asserted an expected count of 13 Git LFS files as 26, without ever running the count.** This was the exact failure it had described an hour earlier: a checkpoint number is worth something only if it came from an independent measurement, and this one came from memory.

5. **Broke a tool's safety check with a safer-sounding instruction.** Removing the `origin` remote straight after cloning meant nothing could accidentally push a rewrite over the original repo, which was the right instinct. But `git filter-repo` reads a missing origin as evidence the repo is not a fresh clone, and refuses to run.

A sixth, smaller: scoping `git status` to a single folder path displayed a rename as an orphaned added file, because rename detection needs both halves of the pair visible at once. It looked like a failure and was a reporting artifact. `git diff --cached -M --name-status` showed R100.

The check that did the work

Every one of those was caught the same way. A number was stated before the command ran, then the command ran and the two were compared.

```
wc -l /tmp/big-exr.txt        # expect 68
```

A verification command with a predicted value attached stops being output and becomes a test. The prediction has to arrive by a different route than the thing it is checking, or it is circular, which is exactly what error 4 above was.

The strongest check of the term was not a number at all. Comparing `git ls-tree -r HEAD --long` between the original repository and the rewritten one, across every path that was kept, gave 1461 files byte for byte identical. Checking against a known-good reference beat every absolute figure that got guessed at.

Definition, since I had to look it up: a **blob** is Git's stored copy of one version of one file's contents. `ls-tree --long` lists every blob at a commit with its size, so comparing the two lists proves the rewrite changed history without changing the current state of the project.

The optimization

One item, chosen because fixing it does two jobs at once.

`PlayerInteraction.OnGUI` built a `new GUIStyle(GUI.skin.box)` and an interpolated string on every pass. `OnGUI` fires more than once per frame, at minimum a Layout pass and a Repaint pass, so that is at least two managed allocations per frame for a text label that changes only when the player walks up to a different object.

The fix caches the style, the `GUIContent` and the measured rect, and rebuilds them only when the label text or the window size changes. The draw path is now one `GUI.Box` call and no allocation.

The first version of that fix cached per interaction target instead of per label, and that was wrong. `ProcessingStation.GetInteractionPrompt` reports how many raw ingredients are queued, so the label changes while the target stays the same, and the prompt froze at whatever it said when the player first walked up. The optimization was real and the code was

broken at the same time, which is the part worth recording. A measurement that says a change is cheaper says nothing about whether it is correct, and the profiler will happily confirm an improvement in a code path that no longer does its job.

No Profiler capture was taken, so there is no before and after number here. The change is structural rather than measured: an allocation that happened every pass now happens once. That is a weaker claim than the one this section was supposed to make, and by the standard set in the section above it, a weaker claim is the correct one to make. A before number and an after number taken the same way is evidence. A description of what the code no longer does is an argument, and an argument is what I have.

The tool that is not here

Part 3 asked for a tool. The plan was a recipe authoring and validation editor window: list the `CombinationRuleData` assets, create new ones from an ingredient picker, and flag duplicate rule sets, missing ingredient references and unset outcome enums.

The validation half is the part that mattered. Recipes match as a multiset, meaning ingredient order does not count and duplicates do, so two rules can be logically identical while looking different in the Inspector. That is not a bug anyone finds by reading the assets. It is the kind of thing a validator finds in a second and a person finds after twenty minutes of confused playtesting.

An earlier draft of the plan had the validator generalised to work across a second asset type. That got cut before any code existed, on the grounds that it was chasing a rubric descriptor rather than solving a problem I had. I still think cutting it was correct, and it is not the reason the tool is missing.

The tool is missing because the loop came first and the loop was the right thing to put first. Nothing about the ordering was wrong. The estimate of what would fit in the remaining time was.

Revisions to the specification, R9 onward

R9. State machine introduced, `IState` implemented for the first time. Changed: `StateMachine.cs` added to `Core/`, four states added under `Gameplay/States/`, `GameManager` added as the context object and the only script that calls `SceneManager`. Alternative considered: a single `GameManager` with an enum and a switch statement, which is fewer files. Trade accepted: more files, in exchange for the driver containing no game knowledge, which is what Part 1's reusability requirement asks for. The enum version would have to be rewritten to be reused; the driver version does not.

R10. Day timer added as the lose condition. Changed: `dayLengthSeconds` on `GameManager`, counted down in `PlayingState`. Alternative considered: an order queue with per-order timers. Trade accepted: one clock instead of many, because the finite ingredient stock

that would have made per-order pressure meaningful did not get built. A per-order timer with infinite ingredients is pressure without a decision attached to it.

R11. Win check placed before the timeout check. Changed: `PlayingState.Update` tests the quota first and returns before touching the clock. Trade accepted: filling the quota on the final frame is a win rather than a tie. Arbitrary, but it has to be decided somewhere, and deciding it in favour of the player is the version that does not feel like a bug.

R12. Restart implemented as a scene reload rather than a reset method. Changed: `GameManager.Restart` calls `SceneManager.LoadScene` on the active build index. Alternative considered: a `ResetForNewDay` method on each system. Trade accepted: a reload is coarser and costs a load, but a system added next term cannot be forgotten in a reset method that nobody remembers to update. Every event listener in the project already unsubscribes in `OnDisable`, so the reload is clean.

R13. Interaction prompt drawing changed from per-pass construction to cached. Covered above.

R14. Spawner objects removed from the game scene. Changed: the six `Decor_Spawner_*` objects are gone from `Main`. Reason: the specification already described them as test tooling excluded from game scenes, so their presence contradicted a document I wrote. Their replacement, a fixed manifest of ingredients delivered at day start, is not built, so ingredient supply is currently whatever is placed in the level by hand.

R15. Input system migration completed. Every legacy `Input` call removed. Changed: `PlayerPickup`, `PlayerInteraction`, `IngredientSpawner` and the new state classes now read `Mouse.current` and `Keyboard.current` from the Input System package. `GameManager` stores its start and restart bindings as `Key` rather than `KeyCode`. Reason: `activeInputHandler` is set to 1, Input System only. This is the part I had wrong. I had been treating the half-finished migration as cosmetic debt, on the assumption that leftover legacy calls would quietly return false. In Input System only mode the legacy `Input` class throws instead. `PlayerPickup` was entirely legacy, so hold-to-carry was throwing on every frame it was pressed. No ingredient could be carried, nothing could be delivered, and the quota was therefore unreachable. Trade accepted: none. This was a bug, not a design choice.

The part worth keeping: this was found by searching the whole project for legacy input calls while wiring something unrelated, not by playing. A pickup that does nothing looks identical from the outside whether the cause is a thrown exception, a wrong layer mask or a missing collider, so playing it would have told me it was broken without telling me why. The same point as the section above, from the other direction: a symptom is not a measurement.

R16. Input components now unsubscribe through a cached reference instead of the singleton. Changed: `PlayerLocomotionInput`, `PlayerActionsInput` and `ThirdPersonInput` capture the `PlayerControls` object in `OnEnable` and use that captured reference in `OnDisable`, rather than asking `PlayerInputManager.Instance` for it a second time. Also removed a `print(MovementInput)` call from the movement callback, which was logging and allocating a string every time the input value changed. Alternative considered: silencing the error log in `OnDisable` and leaving the lookup alone. Trade accepted: none, the cached reference is smaller and correct. Unity destroys scene objects in an

unspecified order, so the manager can be gone before its subscribers shut down. A component that holds its own reference to what it subscribed to can always unsubscribe. One that asks the owner for it again depends on the owner still existing, which during teardown is not a safe assumption.

Why this belongs in the log rather than being a silent fix: the error had been logging for weeks and was ignored, on the correct observation that everything still worked. It stayed correct right up until the restart existed, because a scene reload is a teardown, so an error that fired twice per session was about to fire twice per restart. The lesson is not that ignored errors are always real. It is that “harmless” is a judgment about the current feature set, and it expires quietly when the feature set changes.

What changes in the process next term

- Estimate the tool against the calendar before the loop starts, not after. The ordering was right both times. The estimate was wrong both times, in the same direction.
- Keep the stated-value-then-measure habit and stop treating it as something reserved for risky commands. It cost nothing on the small ones and it caught four errors on the large ones.
- One repository per course. This one carried four courses on branches with the Unity project two folders deep, and the history filtering it needed this week took an evening that the tool could have used.
- Finish a migration in one sitting or write down what is left. The input migration sat half done for most of the term and cost a working pickup. Half a migration is worse than none, because the parts that still work make the parts that do not look like a different problem.
- Write down the reason an error is being ignored, next to the error. The input teardown error was correctly ignored at the time and wrong to keep ignoring once restart existed. A one-line note saying which assumption makes it harmless is enough to notice when that assumption stops holding.
- The player prefab is carrying eleven components, five of which are input or controller scripts from two different tutorial stacks. That is a merge of two working systems rather than a design. It is a refactor for the start of next term, not for the week something is due. Splitting a controller that currently works, under a deadline, is a mistake I have already made once this year on a jam project.